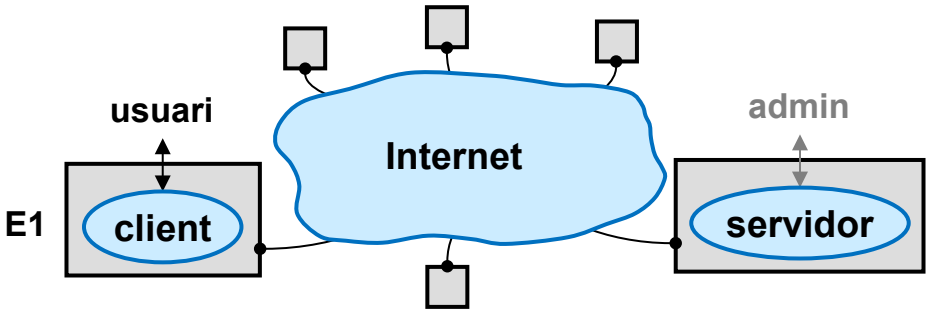
PRÀCTICA 1:
L'APLICACIÓ ECO AMB *SOCKETS* TCP/IP

Enunciat

Presentació



- En aquesta pràctica construireu l'aplicació ECO, un exemple d'una "aplicació en xarxa" (o distribuïda), fent servir *sockets* TCP/IP
  - és una aplicació molt senzilla: retorna una còpia ("eco") del que escriu l'usuari
  - la fareu a partir de la interfície de *sockets*: és una "llibreria" (de fet, *system calls* de l'S.O.) que permet que processos (sobre l'S.O.) puguin comunicar-se entre sí
  - part del codi que fareu us servirà per a l'altra pràctica

- La fareu en parella

- Duració: 5 sessions

- Avaluació a partir d'una demostració individual del funcionament de l'aplicació (a la sessió final) i l'entrega d'un informe per parella (el dia abans de la sessió final)

Requisits (i)

- **R1. Construcció de l'aplicació ECO, seguint un model C-S**
  - **l'usuari escriu línies, el C les envia al S, el S en retorna una còpia ("eco") i el C les mostra**
  - **el C, després de rebre i mostrar l'eco d'una línia rebuda, "torna a l'inici", a esperar que l'usuari escrigui una nova línia o bé que indiqui que vol parlar amb un altre S**
  - **fa servir TCP i és el C qui tanca la connexió TCP**
  - **el S és "iteratiu": serveix peticions una darrera l'altra, no simultàniament, és a dir, només pot mantenir una connexió TCP alhora (no és un S concurrent)**

Requisits (ii)

- **R2. La interfície aplicació-usuari del C (execució en un terminal) ***
  - **l'usuari obre el C i es passa a la situació inicial**
  - **situació inicial: el C pregunta a l'usuari per teclat amb quin S vol parlar**

    l'usuari entra via teclat l'adreça del *socket* del S (@IP i #port TCP)

    el C (crea el seu *socket* en un #port TCP qualsevol i) demana al S la connexió TCP

    es mostren per pantalla les adreces dels 2 *sockets*, @IP i #port TCP, és a dir, la del *socket* local i la del *socket* remot de la connexió TCP establerta entre el C i el S

    l'usuari escriu una 1a línia pel teclat i el C l'envia al S

    es mostra per pantalla el nombre de *bytes* enviats

    el C rep del S l'eco de la línia i la mostra per la pantalla

    es fa el mateix amb una 2a línia, 3a, etc., ...

    ... fins que l'usuari indica que ja no vol enviar més línies escrivint una "línia especial" (la que vulgueu) i que el C no envia al S; llavors el C tanca la connexió

    el C pregunta a l'usuari si vol continuar i tornar a la situació inicial o ...

    ... si vol acabar l'execució

*** podeu fer-hi canvis només per fer les millores, si ho creieu convenient**

Requisits (iii)

- **R3. La interfície aplicació-usuari (administrador) del S (execució en un terminal) ***
  - **l'administrador obre el S, el S pregunta a l'administrador per teclat el #port TCP on esperarà peticions de connexions TCP, i crea el seu *socket* en aquest #port; es passa a la situació inicial**
  - **situació inicial: el S espera la petició de connexió TCP d'algun C i quan n'arriba una, l'accepta i es fa la connexió TCP**

    es mostren per pantalla les adreces dels 2 *sockets*, @IP i #port TCP, és a dir, la del *socket* local i la del *socket* remot de la connexió TCP establerta entre el C i el S

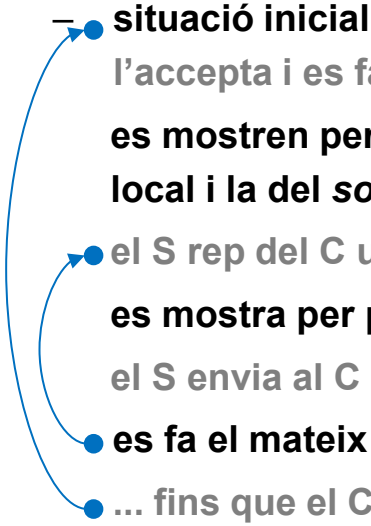
    el S rep del C una primera línia i la mostra per la pantalla

    es mostra per pantalla el nombre de *bytes* rebuts

    el S envia al C aquesta mateixa línia (fa l'eco)

    es fa el mateix amb una segona línia, etc., ...

    ... fins que el C tanca la connexió; llavors el S torna a la situació inicial
  - **l'administrador acaba l'execució del S entrant per teclat ^C (CTRL+C)**



*** podeu fer-hi canvis només per fer les millores, si ho creieu convenient**

Requisits (iv)

- **R4. Programació en llenguatge C i estructura del codi en diversos fitxers, per separar "interfície aplicació-usuari + capa d'aplicació ECO" de la capa de transport TCP**

  fitxers

  - **interfície apl-usuari + capa aplicació: "cliECO.c" i "serECO.c", el main() del C i S ECO amb la part de teclat/pantalla i la part de (construir i enviar missatges) PECO [1] entre C i S**
  - **capa transport: "tTCP.c" (i el .h), la "nova" interfície de la capa TCP, tTCP**

  - **heu de separar "interfície aplicació-usuari + capa d'aplicació" de "capa de transport": a cliECO.c i serECO.c NO hi ha d'haver crides a la interfície de *sockets* TCP, sinó només crides a la "nova" interfície de *sockets*, codi relatiu a (construir i enviar missatges) PECO [1] i codi relatiu a la interfície apl-usuari; a tTCP.c NO hi ha d'haver codi relatiu a la interfície apl-usuari ni a PECO [1], només crides a la interfície de *sockets* TCP**
  - **heu d'implementar la "nova" interfície [2] de la capa TCP, tTCP (la que es dissenyi a classe) [3]**
  - **és l'esquelet #1**

[1] els missatges que C i S s'envien sobre TCP és el protocol de la capa d'aplicació ECO (PECO)

[2] la interfície de la capa són les funcions "externes" del corresponent fitxer ".c"

[3] podeu fer canvis en la interfície (les funcions externes) només per fer les millores, si ho creieu convenient; quant a les funcions "internes" dels fitxers, feu les que us sembli millor

Requisits (v)

- **R5. Estudi dels protocols d'aplicació i transport i dels *sockets* TCP**
  - **feu un estudi dels protocols d'aplicació (PECO *) i transport de l'aplicació ECO fent servir l'analitzador de protocols *Wireshark*:**

    cas d'estudi: s'obren el C i el S en diferents VMs; el C demana la connexió al S i li envia, p.e., 2 línies; s'acaba l'enviament de línies (i per tant es tanca la connexió TCP); es tanquen C i S
  - **feu un estudi dels *sockets* oberts i les seves adreces de l'aplicació ECO fent servir la comanda "ss":**

    cas d'estudi: s'obren el C i el S en diferents VMs; el C demana la connexió al S i li envia, p.e., 2 línies; s'acaba l'enviament de línies (i per tant es tanca la connexió TCP); es tanquen C i S

*** els missatges que C i S s'envien sobre TCP és el protocol de la capa d'aplicació ECO (PECO)**

Requisits (vi)

- **R6. El protocol PECO *: només té PECO LIN i el format és lliure**
  - **el disseny de PECO * ha de complir el següent: només hi ha un missatge, "el missatge d'informació" o PECO LIN, que conté, és clar una línia; no hi ha d'haver altres missatges, p.e., un per iniciar o un per acabar (això s'indica amb la connexió i la desconnexió TCP)**
  - **el format del missatge PECO LIN és lliure excepte en el següent: la línia escrita en el camp informació ha d'estar representada pels *bytes* ASCII dels caràcters de la línia; quant a la resta, feu el que vulgueu, p.e., sense res més, o amb un caràcter salt de línia ('\n') al final, o amb un caràcter nul ('\0') al final, o amb un camp longitud, o amb un/s caràcter/s inicials 'L' o 'L' 'l' 'N', etc.**

*** els missatges que C i S s'envien sobre TCP és el protocol de la capa d'aplicació ECO (PECO)**

Requisits (vii)

- **R7. Descripció de l'aplicació ECO en forma de capes**
  - **heu de descriure l'arquitectura de l'aplicació ECO en forma de capes, cadascuna amb els seus serveis, interfície i protocol (el protocol inclou 3 aspectes, el nom i significat dels missatges, el seu format i la seva seqüència temporal), i a més, a sobre de tot, la interfície entre l'usuari i l'aplicació**

Millores senzilles * (i)

- **MS1. Comprovació que al S, scon i sesc no són descriptors de 2 *sockets* diferents sinó 2 descriptors d'un mateix *socket*, ja que tenen la mateixa @ (@IP i #port TCP)**
  - **al S hi ha 2 descriptors, sesc i scon, on un ve de l'altre via "scon = accept(sesc)": sesc és per a acceptar peticions fent accept() i scon està associat a la connexió per fer-hi read() i write(); no són 2 *sockets*, sinó 2 descriptors del mateix *socket*, ja que tenen igual @IP i #port TCP**
  - **per comprovar-ho, al S feu getsockname() sobre scon i sesc, i pels dos mostreu per pantalla @IP i #port TCP, en aquest cas:**

    Cas 1: no feu "bind(0.0.0.0)" al S, és a dir, a bind(sesc) no poseu l'@IP 0.0.0.0 sinó la de la interfície *Ethernet* eth1 (p.e., 82.2.3.4); obriu el C i feu que es connecti al S
  - **I si a bind(sesc) poseu l'@IP 0.0.0.0, llavors l'@IP de scon al S també serà 0.0.0.0?**

    (Ei, recordeu que l'@IP 0.0.0.0 equival a "totes les @IP" de la màquina: a la VM seria l'@IP de la interfície *Ethernet* eth1, p.e., 82.2.3.4, i l'@IP de la interfície de *loopback* lo, 127.0.0.1; fer "bind(0.0.0.0)" és igual a fer alhora "bind(82.2.3.4)" i "bind(127.0.0.1)", i així el S escolta peticions de connexió dirigides a "totes les @IP", a qualsevol, a 82.2.3.4 i a 127.0.0.1)

    La resposta és "sí" però amb un matís: l'@IP d'scon al S no serà 0.0.0.0 sinó una de concreta de la màquina, la d'eth1 (p.e., 82.2.3.4) o la de lo, 127.0.0.1; però de fet 0.0.0.0 són totes les de la màquina, o sigui que la resposta és "sí". Per comprovar-ho, al S feu getsockname() sobre scon i sesc, i pels dos mostreu per pantalla @IP i #port TCP, en aquests 2 casos:

    Cas 2: feu "bind(0.0.0.0)" al S; a una VM diferent de la del S, obriu el C i feu que es connecti a l'@IP (i #port TCP) del S d'eth1 (p.e., a 82.2.3.4)

    Cas 3: feu "bind(0.0.0.0)" al S; a la mateixa VM del S, en un altre terminal, obriu el C i feu que es connecti a l'@IP (i #port TCP) del S de lo, 127.0.0.1

*** si per fer les millores heu de canviar la interfície aplicació-usuari o la "nova" interfície de la capa TCP, ho podeu fer**

Millores senzilles * (ii)

- **MS2. El protocol PECO actual no té en compte que TCP és "orientat a flux" i no separa bé línies consecutives: estudiar-ho i arreglar-ho**
  - **en el disseny del protocol PECO hi ha una cosa mal feta: hem suposat que és un protocol "orientat a missatge" (com p.e., UDP) però TCP és un protocol "orientat a flux"**

    1 TCP_Envia() pot correspondre a uns quants TCP_Rep() i al revés, diversos TCP_Envia() poden correspondre a 1 TCP_Rep (i potser a diversos paquets TCP)
  - **primer, estudieu aquest problema fent canvis en el codi i unes proves:**
    1) **(si és el cas) feu que la línia s'enviï sense el caràcter salt de línia ('\n') al final**
    2) **feu que el C en lloc de fer 1 cop "llegir teclat + enviar a S + rebre de S" ara faci 3 cops seguits "llegir teclat + enviar a S" i després 1 cop "rebre de S"**
    3) **feu que que C i S, abans de rebre amb TCP_Rep(), s'esperin p.e. uns 20 segons (podeu fer-ho amb la funció sleep(), una crida al sistema), com si estiguessin "enfeinats"**
    4) **feu aquesta prova: en el C, envieu 3 línies (p.e., "Hola1", "Hola2, "Hola3") "ràpidament" (dins d'aquell retard de 20s); heu de veure que al S i a l'eco rebut no es separen les línies**
  - **segon, arregleu aquest problema, redissenyant el protocol PECO i fent canvis en el codi**
    1) **al format del missatge PECO LIN afegiu-hi un "delimitador", és a dir, uns *bytes* que permetin a la capa d'aplicació saber quan acaba el missatge; p.e., el caràcter salt de línia ('\n') o el caràcter nul ('\0') al final, o un camp longitud (en *bytes* de la línia) al principi**
    2) **feu que quan el C i el S reben, busquin el "delimitador" per saber quan acaba la línia, i que vagin llegint i llegint contínuament fins a rebre tots els *bytes* de la línia**
    3) **repetiu la prova anterior i comproveu que ara sí es separen bé les línies**

*** si per fer les millores heu de canviar la interfície aplicació-usuari o la "nova" interfície de la capa TCP, ho podeu fer**

Millores senzilles * (iii)

- **MS3. Obtenció del missatge d'error (i d'errno) de la interfície de *sockets* en 3 casos**
  - **dieu quin és el text del missatge d'error del S.O. i el valor de la variable errno quan feu servir crides a sistema de la interfície de *sockets* en els 3 casos següents:**

    Cas 1: en el bind() del S, no poseu l'@IP 0.0.0.0 sinó una @IP que no sigui cap de la màquina (p.e., si la interfície *Ethernet* eth1 té 82.2.3.4 i la interfície de *loopback* lo té 127.0.0.1, doncs escolliu-ne una altra); llavors, obriu el S

    Cas 2: obriu un primer S; en un altre terminal de la mateixa màquina obriu un segon S en el mateix #port TCP que el primer S

    Cas 3: obriu el C i feu que es connecti a un S "inexistent" (a una @IP i #port TCP on cap S escolta, p.e., tanqueu el vostre S i proveu de connectar-vos-hi, o d'altres maneres)

- **MS4. Acabar l'execució del S "suaument" amb ^C (CTRL+C)**
  - **que quan l'administrador acabi l'execució del S entrant per teclat ^C (CTRL+C), s'acabi "suaument" (recollir *signal*, tancar *sockets*, etc.)**

- **Altres millores senzilles que proposeu vosaltres**

*** si per fer les millores heu de canviar la interfície aplicació-usuari o la "nova" interfície de la capa TCP, ho podeu fer**

Avaluació: demostració i informe (i)

- **Fareu una demostració del funcionament de l'aplicació:**
  - **a la sessió final, a classe, demostració individual**
  - **proves de funcionament i els seus resultats, captura dels paquets generats i la seva anàlisi amb *Wireshark* i preguntes sobre el codi**
  - **omplireu un document d'avaluació de la demostració que entregareu al final de la classe**

- **Entregareu un informe:**
  - **el dia abans de la sessió final, un per parella**
  - **a través del *Moodle* de l'assignatura (només un per parella de 2 estudiants), en un únic fitxer en format ZIP, el nom del qual tingui els vostres dos noms (Cognom1Nom1-Cognom2Nom2-PRA1.zip, p.e., VilaPere-FabregaLluis-PRA1.zip), amb el següent:**
    a) **Els fitxers de codi font de l'aplicació ECO ("p1-cliECO.c", "p1-serECO.c", "p1-tTCP.c" i ".h").**
    b) **Els fitxers de captura de paquets filtrats corresponent a l'estudi dels protocols d'aplicació i transport amb *Wireshark*, cas que expliqueu al document "memòria" de l'informe. Feu servir el format libpcap (.pcap), i no hi guardeu tots els paquets capturats sinó només els necessaris (els "filtrats") per així reduir-ne la grandària (opció "*Displayed*" al menú "*File* → *Save*"; en algunes versions es troba a "*File* → *Export specified packets*...").**
    c) **Un document "memòria" (en format PDF) que inclogui el següent →**

Avaluació: demostració i informe (ii)

c) **Un document "memòria" (en format PDF) que inclogui el següent:**

- **Autors**
- **Requisits i millores**
- **L'arquitectura en capes**
- **La interfície aplicació-usuari**
- **Els serveis de les capes**
- **Les interfícies de les capes**
- **Els protocols de les capes (estudi amb *Wireshark*)**
- **Els *sockets* TCP de l'aplicació (estudi amb "ss")**
- **Les millores**
- **Problemes i suggeriments**
- **Treball en parella i dedicació**
- **Bibliografia**

Teniu al *Moodle* una plantilla .docx per escriure la memòria, amb tots els detalls de com ha de ser

Teniu al *Moodle* una plantilla .pptx per dibuixar seqüències temporals

Avaluació: demostració i informe (iii)

DEMOSTRACIÓ (individual)
Si la demostració està “prou bé”, s’avalua l’informe; en cas contrari, la nota és < 4.0

INFORME (per parella)	
Concepte	**Punts**
Requisits	≤ 6.0
Si els requisits estan “prou bé” (≥ 4.0), s’avalua la resta; en cas contrari, la nota és < 4.0	
Millores *	≤ 3.0
Qualitat de l’informe	≤ 0.5
Correcció i claredat codi	≤ 0.5
Total	≤ 10.0

* 0.8 punts per millora senzilla (fins a un màxim de 3.0)